252-0027

Introduction to Programming

Exercises

Week 5: Arrays, methods, Debugger

Jan Leonard
Department of Computer Science
ETH Zurich

Plan for today

- Arrays
- Signatures
- Value vs. Reference
- Recursion
- Kahoot

Organization

My name: *Jan Leonard*

For questions: leonardj@ethz.ch

- Please put «[EProg26]» in the subject

New exercises: **Tuesday evening** (normally)

Submit exercises by **Tuesday evening (23:59)** following week

- Always submit via Git
- Solutions in a separate Git project

Exercise 6: Substring-Counter (Bonus!)

This exercise gives bonus points!

You can find the rules here: https://lec.inf.ethz.ch/infk/eprog/2026/exercises/additionals/bonusaufgaben_regeln.pdf

First Bonus

- **Java 25**
- **It must compile ! Last push before Deadline counts !**
  - Be aware of output (Cases and other print statements)
- **Do not change class or method names**
- **Do not use static attributes**

Timed Bonus

- **One Bonus exercise every week**
- **Week 7 (03.11) and 12 (08.12) Bonus tasked will be released at 5pm and the Deadline will be at 7 pm**
- **More on that later**

1D Arrays

One-dimensional Arrays

- Arrays occupy a fixed size n in memory and therefore also have a fixed length
- for each entry, a value of the declared data type can be stored
- we can access a specific element i with arr[i]
- indexing starts at 0, so it goes up to n-1

```java
public class Main {
    public static void main(String[] args){
        int[] arr = {3,5,6,1,2,8};
        for(int i = 0; i < arr.length; i++){
            arr[i] *= 2;
        }
    }
}
```

2D Arrays

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

[[1,2,3],[4,5,6],[7,8,9]]

Two-dimensional Arrays

- A matrix could be represented as a two-dimensional array
- int[][] arr = new int[n][n];
  - defines an array that stores n integer arrays of length n (nxn matrix)
- so we obtain an entire row with arr[i] for 0<=i<n
- we obtain an entry with arr[i][j] for 0<=i,j<n
- Caution: the lengths of the arrays could differ!

```java
public class Main {
    public static void main(String[] args) {
        int[][] arr = {{1, 2, 3},{4, 5, 6},{7, 8, 9}};
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                arr[i][j] *= 2;
            }
        }
    }
}
```

2D Arrays

array = {arrOne, arrTwo, arrThree}

i	array[i]
0	
1	
2	

i	0	1	2	3	4	5
arrOne[i]	4	501	-1	200	42	0

i	0	1	2
arrTwo[i]	11	21	-170

i	0	1	2	3
arrThree[i]	100	1	-321	3

```
int[] arOneD = {1,2,3,4,5,6};
```

i	0	1	2	3	4	5
arOneD[i]	1	2	3	4	5	6

```
int[][] arTwoD = {
        {1,2,3},
        {4,5,6},
        {7,8,9}
};
```

i	0	1	2
arTwoD[i]			

i	0	1	2
[i]	1	2	3

i	0	1	2
[i]	4	5	6

i	0	1	2
[i]	7	8	9

```
int[][] arTwoD = {
        {1,2,3},
        {4,5,6},
        {7,8,9}
};
```

arTwoD[i] =

i	
0	{1,2,3}
1	{4,5,6}
2	{7,8,9}

arTwoD[i][j] =

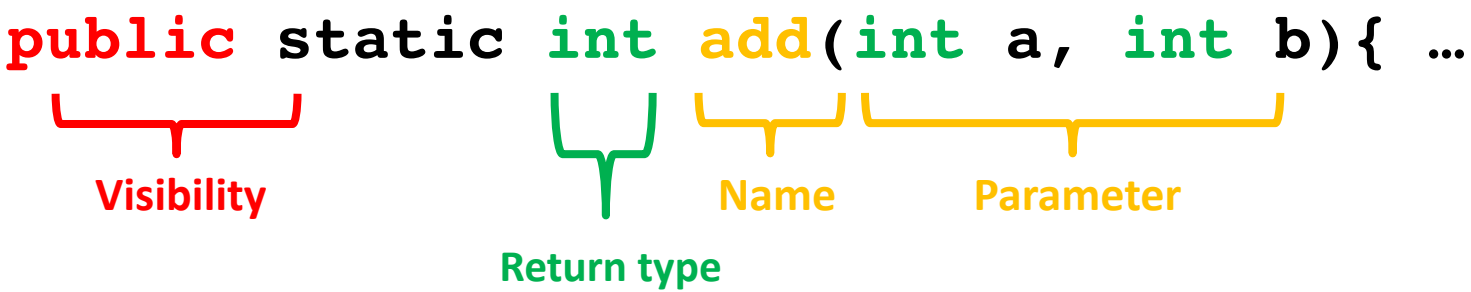
i\j	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

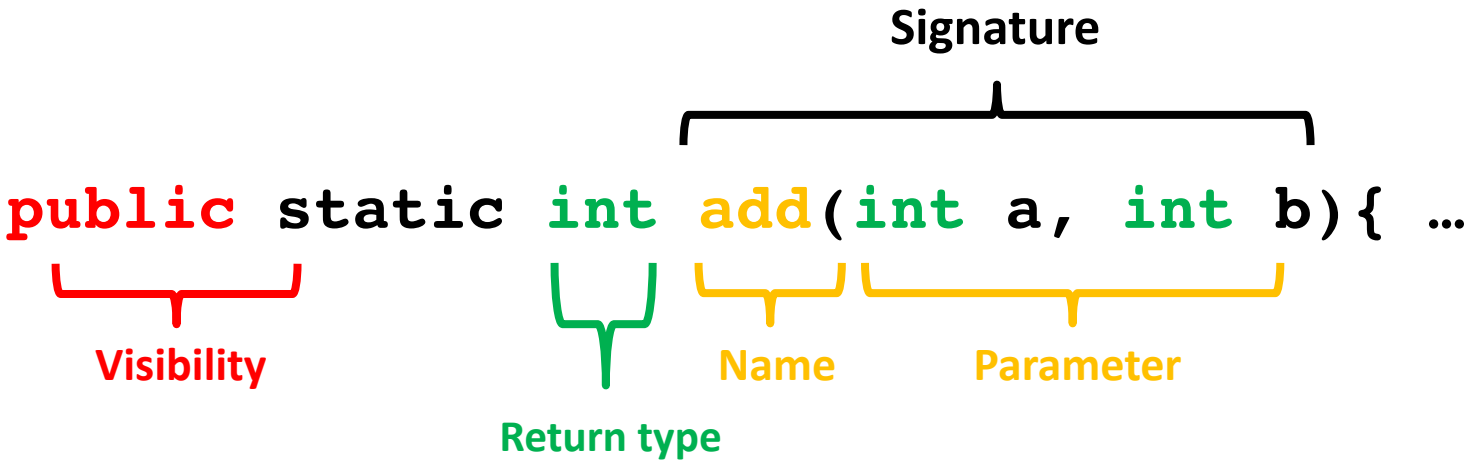
Useful Array Functions

- **Arrays.toString(a)** - returns a string with the elements of a 1D array: [1,2,3]
- **Arrays.deepToString(a)** - returns a string for multidimensional arrays
- **Arrays.equals(a, b)** - checks if two 1D arrays are equal element by element
- **Array.deepEquals(a, b) -** same as .equals(a, b) but for multidimensional arrays
- **Arrays.sort(array)** - sorts the array in ascending order

methods

```
public static int add(int a, int b){ …
```


public static int add(int a, int b){ …
Visibility
Return type
Name
Parameter


Signature
public static int add(int a, int b){ …
Visibility
Return type
Name
Parameter

Method Overloading

- Multiple methods may have the same name.
- The methods must differ in their signature, e.g., in the number, data types, or order of the parameters
- Based on the arguments passed, Java automatically selects the appropriate method.

```java
public class Main {
    // Erste Version: Addiert zwei int-Werte
    public static int add(int a, int b) {
        return a + b;
    }
    // Überladene Version: Addiert drei int-Werte
    public static int add(int a, int b, int c) {
        return a + b + c;
    }
    // Überladene Version: Addiert zwei double-Werte
    public static double add(double a, double b) {
        return a + b;
    }
}
```

Value vs Objects

Objects 101

- classes are "blueprints" that define how data is stored
- Objects are concrete realizations of the class
- The object's variable stores a pointer to where the data is located in memory (reference)

```java
public class Main {
    public static void main(String[] args){
        String s = new String("hello");
        System.out.println(s);
    }
}
class String { … }
```

Primitive types

- variables store the actual value
- are limited in size
- default values are fixed
- Can never be NULL

Reference types

- variables store a reference to an object or null
- objects may require different amounts of memory
- the default value is null (refers to no object)

Passing References

- The variable numbers stores the reference to where the actual values are located
- pass the reference to modifyArray
- modifyArray finds values in memory, doubles them, terminates
- main looks in memory using the same reference and finds doubled values without us having to return them

```java
public class ReferenceArrayDemo {
    public static void main(String[] args) {
        // Erstelle ein Array
        int[] numbers = {1, 2, 3, 4, 5};
        System.out.println("Vor der Änderung:");
        System.out.println(Arrays.toString(numbers));
        modifyArray(numbers); // Übergabe des Arrays
        System.out.println("Nach der Änderung:");
        System.out.println(Arrays.toString(numbers));
    }
    public static void modifyArray(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            arr[i] *= 2; // Verdopple jeden Wert im Array
        }
    }
```

```java
class Box {
    int value;

    Box(int value) {
        this.value = value;
    }
}

public class Demo {
    public static void main(String[] args) {
        int a = 5;
        int b = a;
        b = 7;

        Box p = new Box(1);
        Box q = p;
        q.value = 2;
        q = new Box(3);

        Box r = null;
        r = p;
        p = null;
        System.out.println(r.value);
        System.out.println(p.value);
    }
}
```

2

Exception

Recursion



Eprog TA — HS 2026

janleonard115.github.io

Recursion - Example

1. Implementieren Sie die Methode `Calculations.checksum(int x)`, das heisst die Methode `checksum` in der Klasse `Calculations`. Die Methode nimmt einen Integer `x` als Argument, welcher einen nicht-negativen Wert hat. Die Methode soll die Quersumme von `x` zurückgeben. Sie sollen für diese Aufgabe **keine** Schleife verwenden.

   Beispiele

   - `checksum(258)` gibt 15 zurück.
   - `checksum(49)` gibt 13 zurück.
   - `checksum(12)` gibt 3 zurück.

   Hinweis: Für einen Integer a ist `a % 10` die letzte Ziffer und `a / 10` entfernt die letzte Ziffer. Zum Beispiel `258 % 10` ist 8 und `258 / 10` ist 25.

Exercise 1: Sum of Elements

Write a method

```
int sum(int[] xs)
```

that computes the sum of all elements of an array.
Examples: `[]` $\rightarrow 0$
`[7]` $\rightarrow 7$
`[2, 5, 1]` $\rightarrow 8$
Hint: Use a helper method that carries the current index along. This lets you build up the sum step by step without a loop.

Exercise 2: Counting Characters

Implement

```
int count(char c, String s)
```

which counts how often the character `c` occurs in the string `s`. Use `s.substring(1)` to recurse over the rest of the string.
Examples:
`count('a', "banana")` $\rightarrow 3$
`count('x', "")` $\rightarrow 0$

Exercise 3: Sorted Array

Implement

```
boolean isSorted(int[] xs)
```

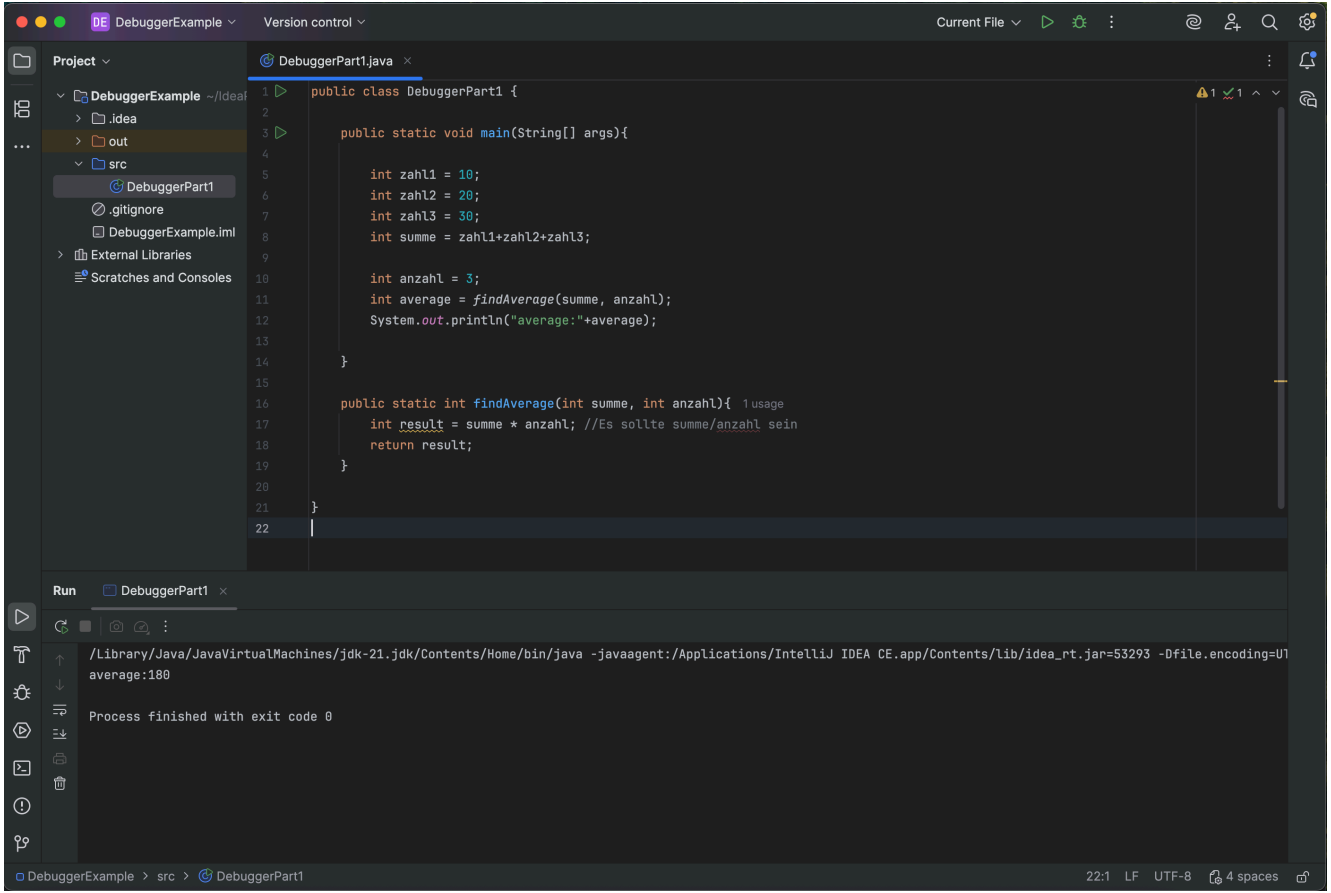
which checks whether the array `xs` is sorted in **non-decreasing order**.
Examples:
`[]` → `true`
`[5]` → `true`

`[1,2,2,5]` → `true`
`[2,1]` → `false`
Hint: Use a helper method with an index parameter to compare the next pair of elements.

Java Debugger

What is the debugger and what does it do?

- A tool (in Java) that helps with debugging.
- With the Java debugger, you can go through the program step by step and observe exactly how it is executed.
- The changes to the variables and their values in each line are shown in a table.



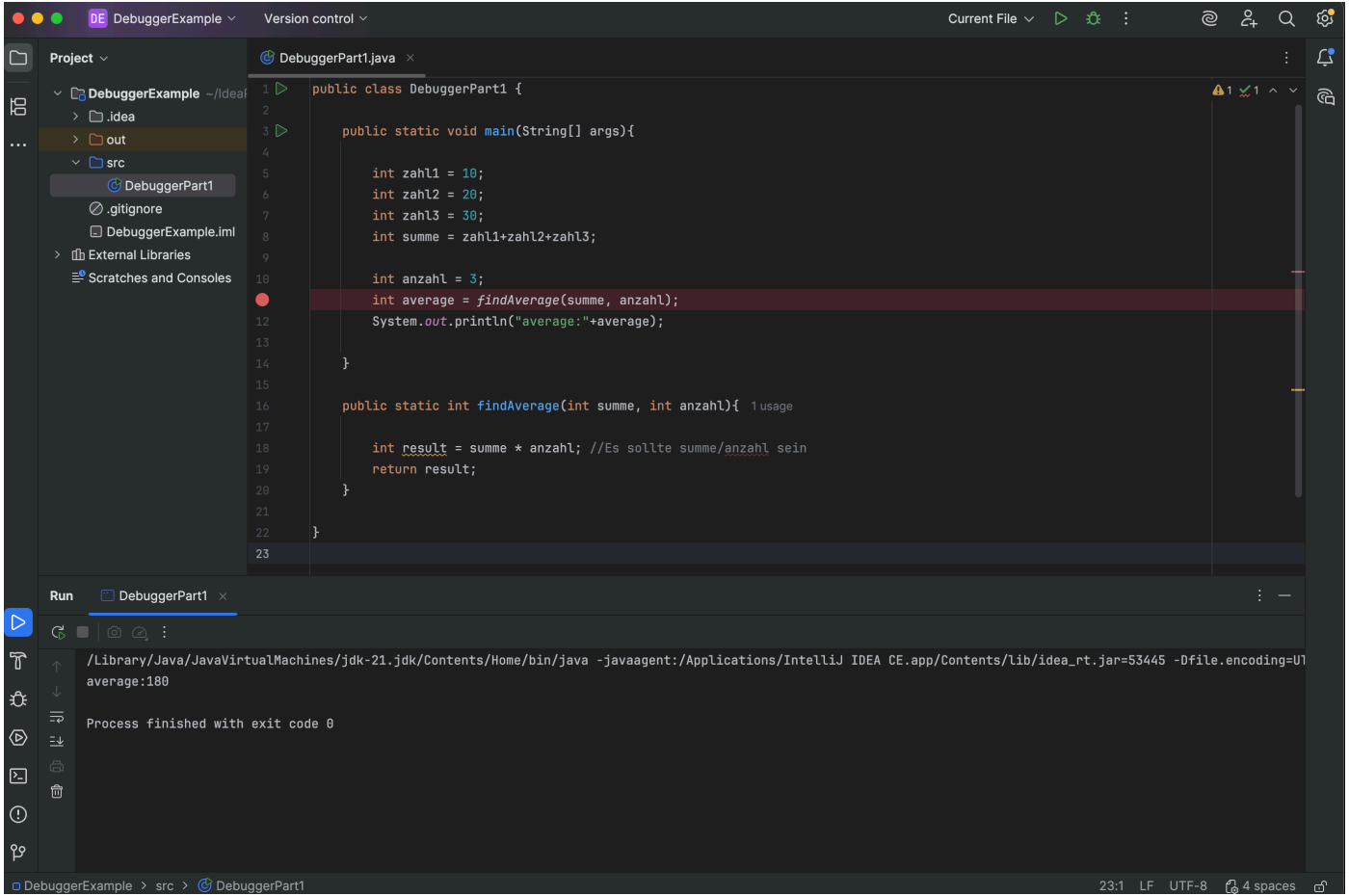
The program is not doing what you expect, and you cannot find the errors?

Use the debugger!

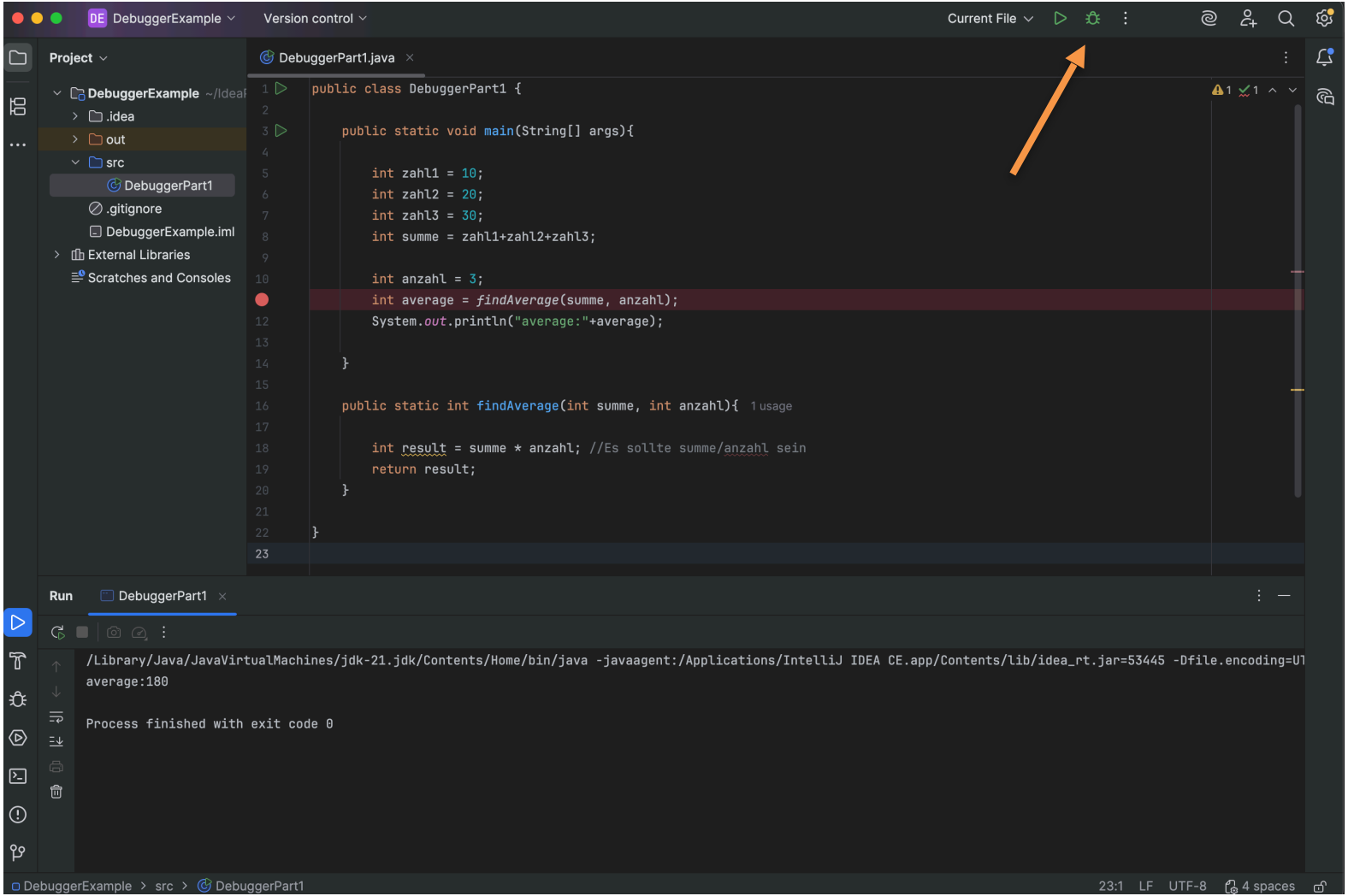
Breakpoints

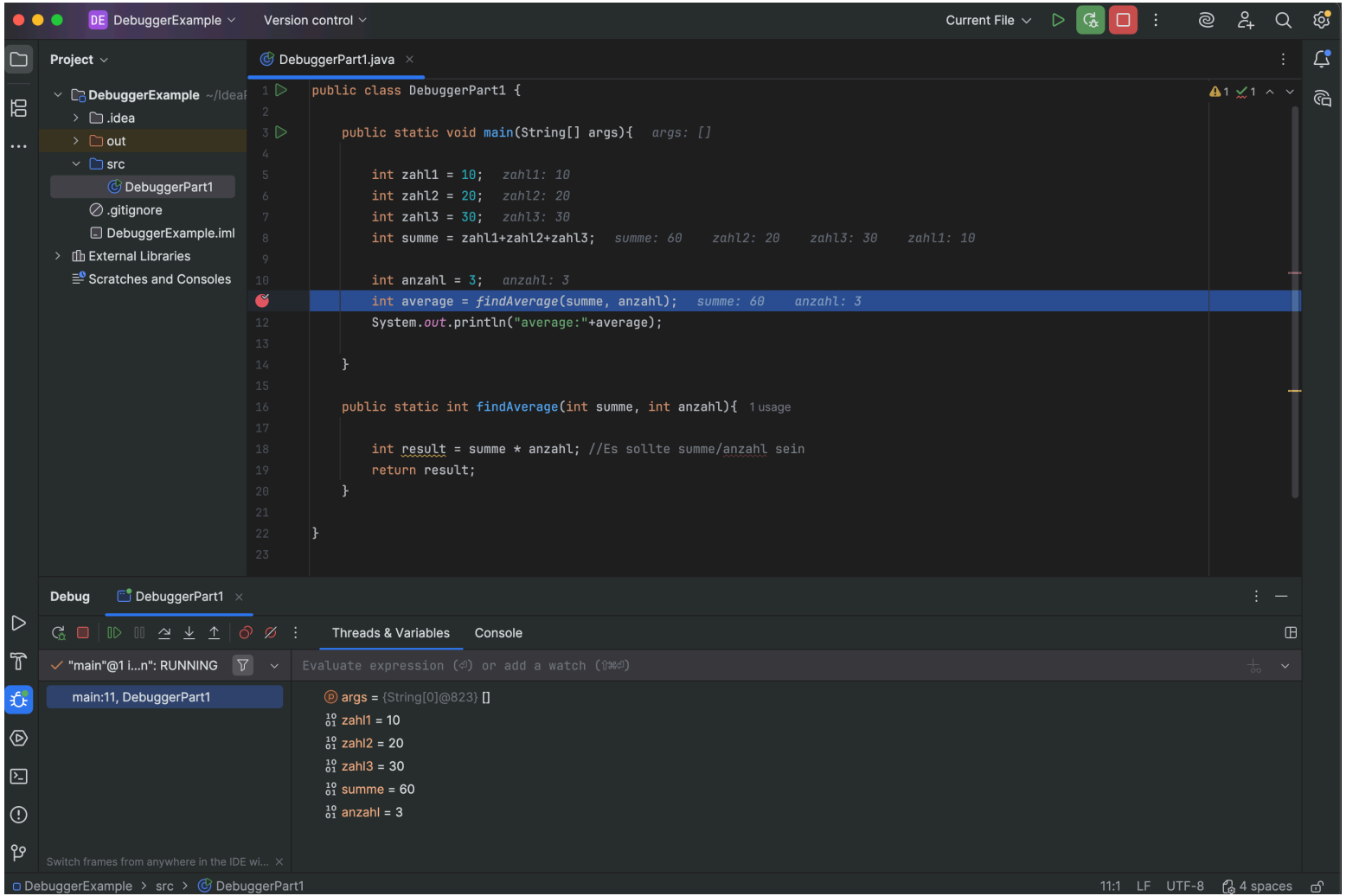
- Everything is executed automatically up to the breakpoint, and as soon as the line with the breakpoint is reached, automatic execution stops.
- After that, the user can execute the program line by line themselves.
- Choosing a breakpoint: At a point where you know that everything up to there works as expected.
- Setting a breakpoint: Click to the left of the column with line numbers to set the breakpoint



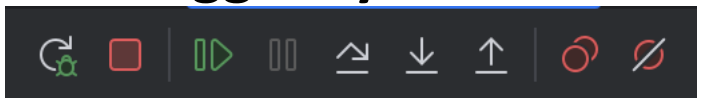


- Since we are sure that summe and anzahl are correct, we now want to test the method findAverage.
- For this, we set a breakpoint in the line where this method is called.


DebuggerExample
Version control
Current File
Project
DebuggerExample ~/Ideaf
.idea
out
src
DebuggerPart1
.gitignore
DebuggerExample.iml
External Libraries
Scratches and Consoles
DebuggerPart1.java
public class DebuggerPart1 {
public static void main(String[] args){
int zahl1 = 10;
int zahl2 = 20;
int zahl3 = 30;
int summe = zahl1+zahl2+zahl3;
int anzahl = 3;
int average = findAverage(summe, anzahl);
System.out.println("average:"+average);
}
public static int findAverage(int summe, int anzahl){ 1 usage
int result = summe * anzahl; //Es sollte summe/anzahl sein
return result;
}
}
Run
DebuggerPart1
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib/idea_rt.jar=53445 -Dfile.encoding=U
average:180
Process finished with exit code 0
DebuggerExample > src > DebuggerPart1
23:1 LF UTF-8 4 spaces


DebuggerExample
Version control
Current File
Project
DebuggerExample ~/IdeaP
.idea
out
src
DebuggerPart1
.gitignore
DebuggerExample.iml
External Libraries
Scratches and Consoles
DebuggerPart1.java
1 public class DebuggerPart1 {
2
3 public static void main(String[] args){ args: []
4
5 int zahl1 = 10; zahl1: 10
6 int zahl2 = 20; zahl2: 20
7 int zahl3 = 30; zahl3: 30
8 int summe = zahl1+zahl2+zahl3; summe: 60 zahl2: 20 zahl3: 30 zahl1: 10
9
10 int anzahl = 3; anzahl: 3
11 int average = findAverage(summe, anzahl); summe: 60 anzahl: 3
12 System.out.println("average:"+average);
13
14 }
15
16 public static int findAverage(int summe, int anzahl){ 1 usage
17
18 int result = summe * anzahl; //Es sollte summe/anzahl sein
19 return result;
20 }
21
22 }
23
Debug
DebuggerPart1
Threads & Variables
Console
"main"@1 i...n": RUNNING
Evaluate expression (⏎) or add a watch (⇧⌘⏎)
main:11, DebuggerPart1
args = {String[0]@823} []
zahl1 = 10
zahl2 = 20
zahl3 = 30
summe = 60
anzahl = 3
Switch frames from anywhere in the IDE wi...
DebuggerExample > src > DebuggerPart1
11:1 LF UTF-8 4 spaces

- **Blue line: everything up to, but not including, this line has been executed**
- **Debugger Symbole:**



View breakpoints
- Alle breakpoints anzeigen

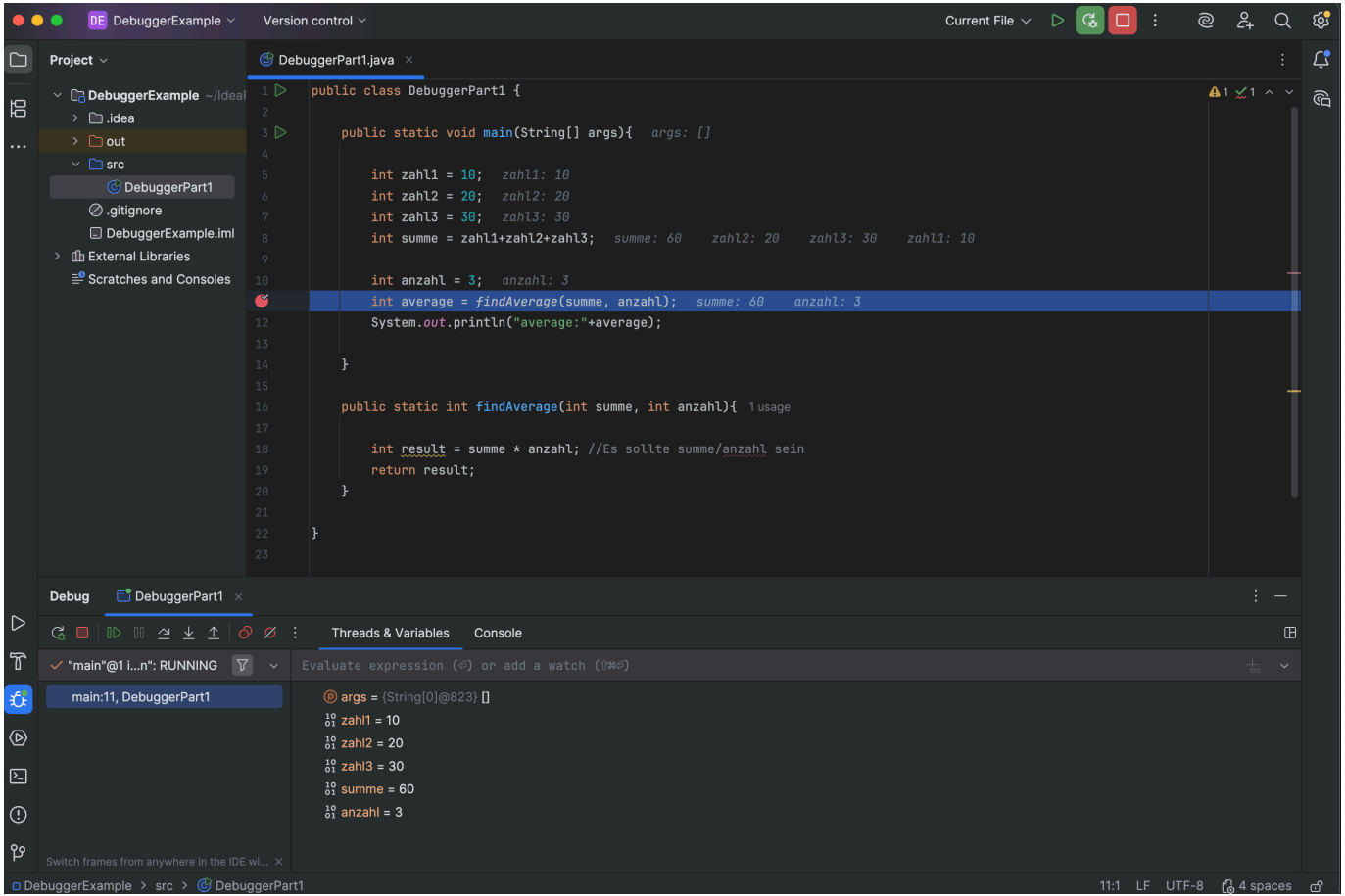
Step out
- To step out of a method

Step into
- To jump into a method.

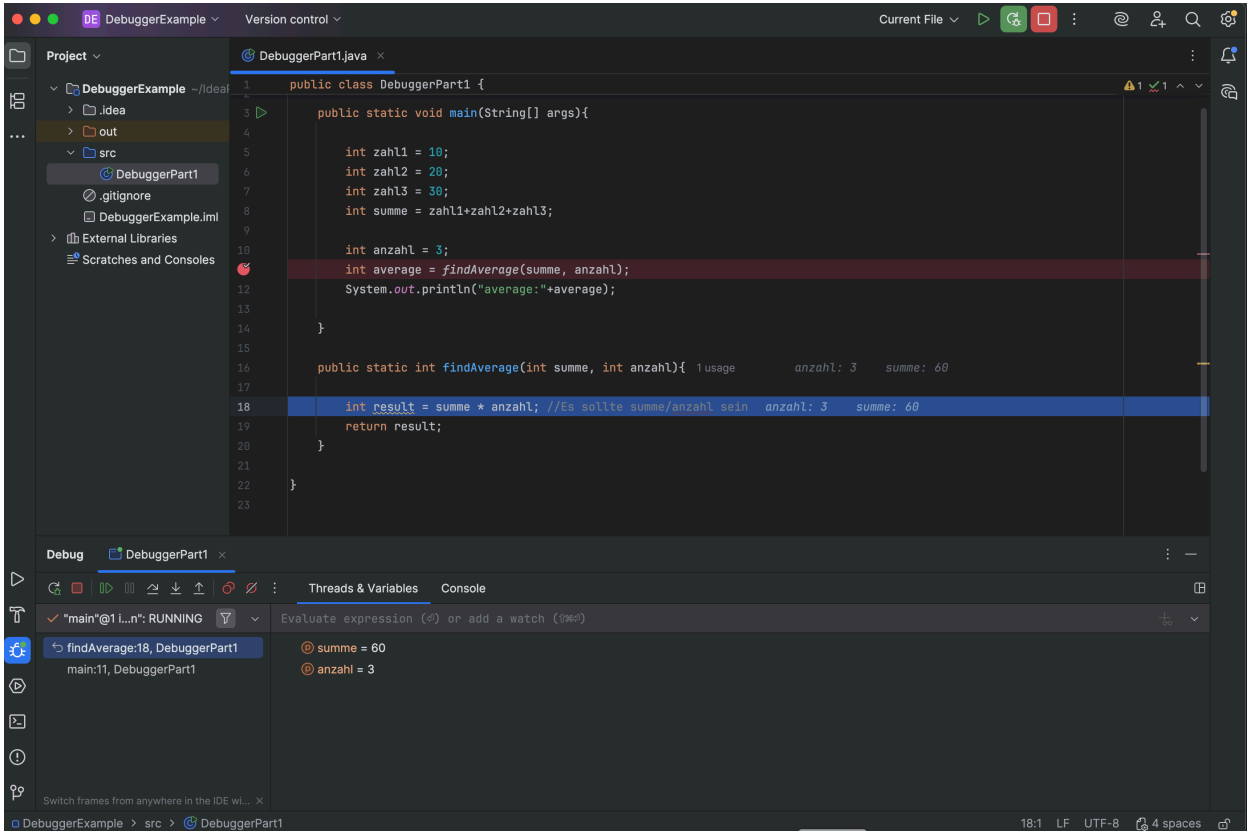
Step over
- To go to the next line.

Termination
- To stop the debugger

Go on from this line…

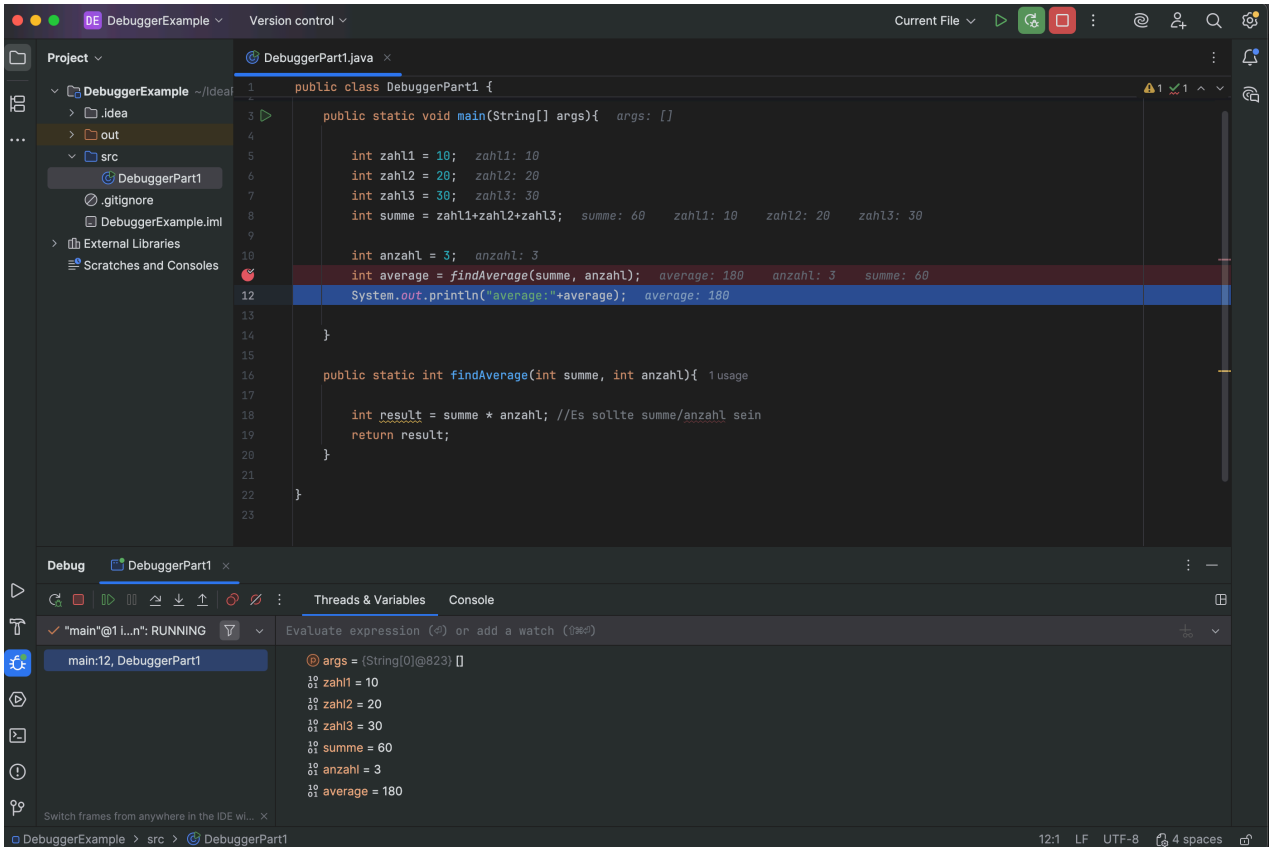


With step into:



We stepped into the called method findAverage and can now execute this method step by step.

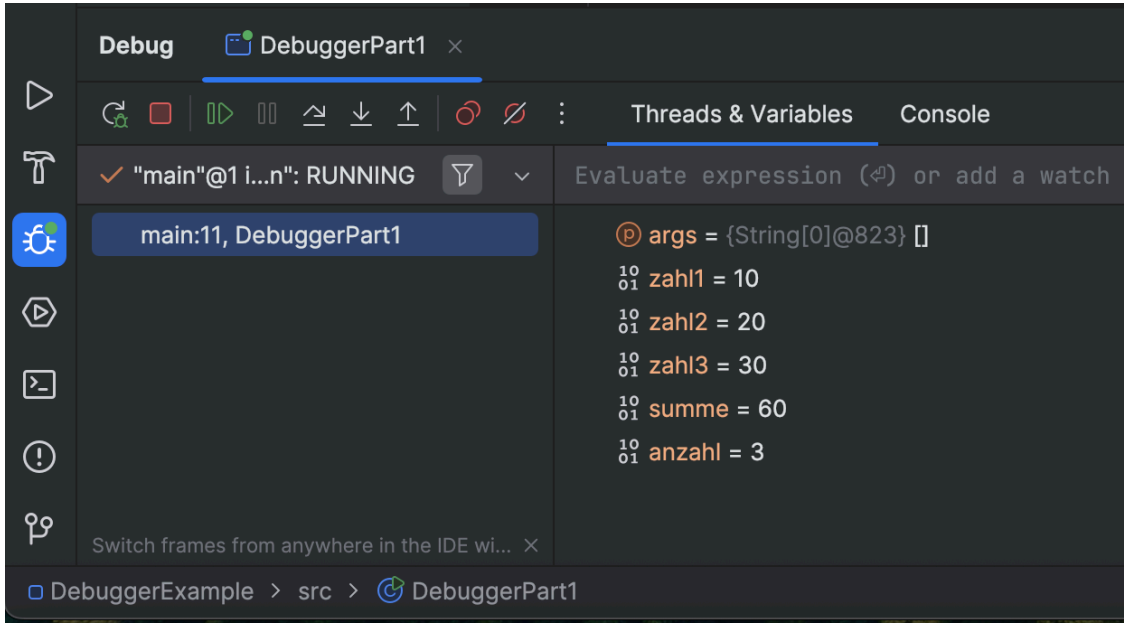
With step over:



- The method call of findAverage was executed in one step and the return value was assigned to average.
- We are now in the next line.

Step over vs. Step into: When to use what?

- **Step over:** 
  - Jump to the next line.
  - If a method is called in the line, it is executed.
  - We assume that it works as expected and that the return value is correct.
- **Step into:** 
  - Go into the method to step through it.
  - Use this when you are unsure whether the method works as expected and you want to inspect the execution more closely.



- Shows all variables and their values that are in the scope of the blue line.
- After each line is executed, you can check whether the values changed as expected.

```java
public class DebuggerPart1 {

    public static void main(String[] args){   args: []

        int zahl1 = 10;   zahl1: 10
        int zahl2 = 20;   zahl2: 20
        int zahl3 = 30;   zahl3: 30
        int summe = zahl1+zahl2+zahl3;   summe: 60    zahl2: 20    zahl3: 30    zahl1: 10

        int anzahl = 3;   anzahl: 3
        int average = findAverage(summe, anzahl);   anzahl: 3    summe: 60
        System.out.println("average:"+average);

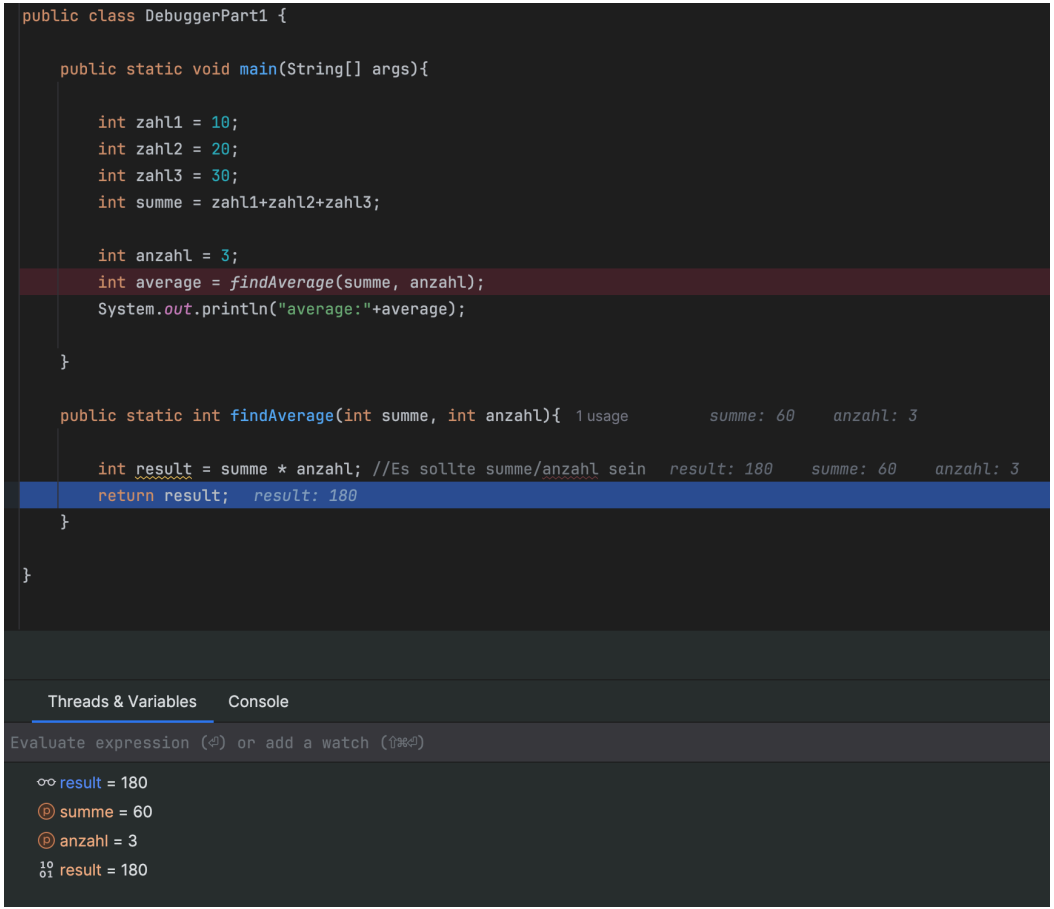
    }

    public static int findAverage(int summe, int anzahl){  1 usage

        int result = summe * anzahl; //Es sollte summe/anzahl sein
        return result;
    }

}
```

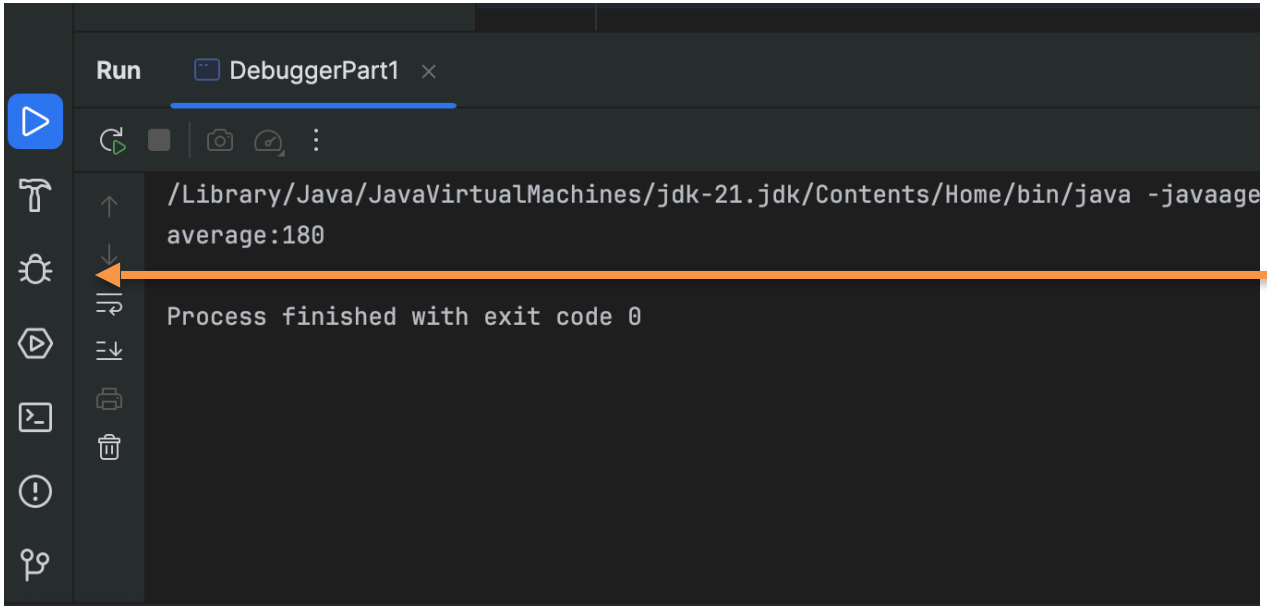
- The values of the variables are also shown in gray at the end of each line



Here you can see which values summe, anzahl, and average contain.

Under "Threads & Variables", you can see that average is computed by (summe * anzahl) instead of by (summe / anzahl).

If Threads & Variables is not shown automatically:



Select
Debug

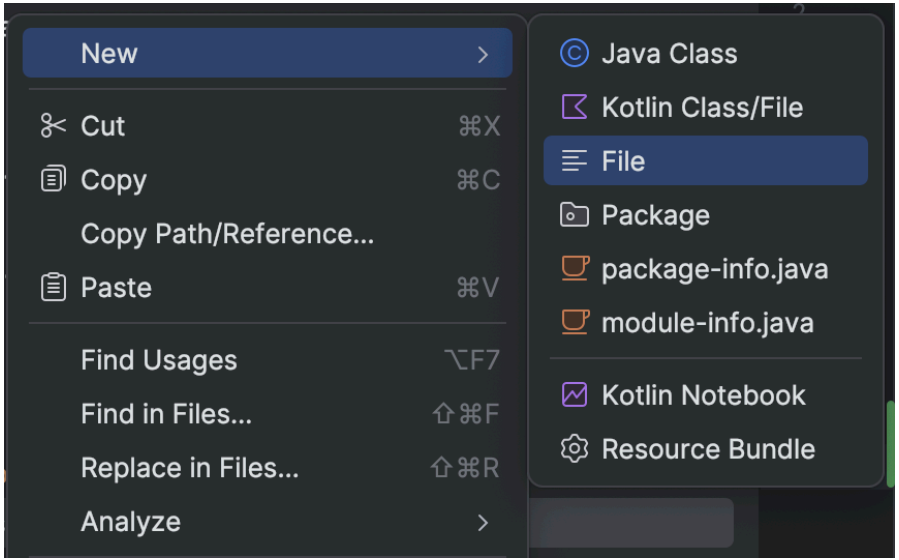
Feedback

Feedback

- Starting with u05, you tell the TAs where you would like feedback.
- For this, create a requestfeedback.txt file in the uXX folder, not in src / test / resources.
- In the file, write the exercises for which you want feedback. (Bonus is graded separately)
- If you want I will also give you feedback on the bonus task.
  - Just write that in the file

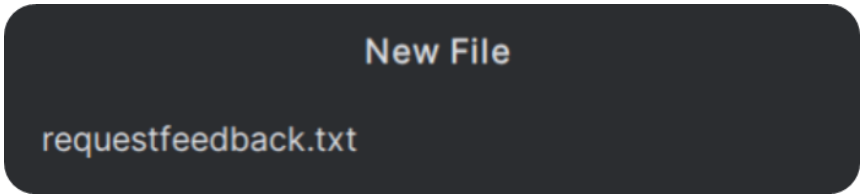
Feedback Datei erstellen

1. **Right-click project folder uXX.**
2. **New -> File.**



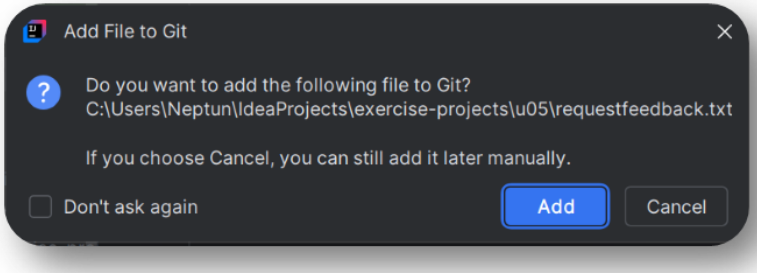
Feedback Datei erstellen

1. **Right-click project folder uXX.**
2. **New -> File.**
3. **Name the file requestfeedback.txt in the window that appears.**



Feedback Datei erstellen

1. Right-click project folder uXX.
2. New -> File.
3. Name the file requestfeedback.txt in the window that appears.
4. Choose Add in the appearing "Add File to Git" window



Review

Exercise 1: Binary Representation

```java
/*
 * Bestimmt exp so, dass 2^exp <= number und 2^(exp + 1) > number gilt.
 */
public static int largestExponent(int number) {  1 usage   zelleraa

    int largestPowerOfTwo = 1; // 2^0 = 1
    int exp = 0;

    if(number <= 0) {
        System.out.println("Keine positive ganze Zahl!");
    } else {
        while(largestPowerOfTwo <= number) { //Erhöhe exp
            exp = exp + 1;
            largestPowerOfTwo = largestPowerOfTwo * 2; //2^exp
        }
        // largestPowerOfTwo > number gilt hier aber es gilt 2 ^ (exp - 1) <= number
        // deshalb gehen wir einen Schritt zurück
        largestPowerOfTwo = largestPowerOfTwo / 2;
        exp = exp - 1;
    }

    return exp;
}
```

Step 1: Find exp such that 2^exp <= number and 2^(exp + 1) > number.

Exercise 1: Binary Representation

```java
/*
 * Gibt die Binaerdarstellung von number aus gegeben dem grössten exp,
 * wo 2^exp <= number und 2^(exp + 1) > number gilt und
 */
public static String binaerDarstellung(int number, int exp, int largestPowerOfTwo) {  1 usage  zelleraa
    // Dummy variables to make code more readable
    int currentPowerOfTwo = largestPowerOfTwo;
    int remainingNumber = number;
    int currentExp = exp;

    String bDarstellung = "";

    while(currentExp >= 0) {
        // Prüfe ob verbleibende Zahl grösser (1) oder kleiner (0) als die currentPowerOfTwo ist
        int digit = remainingNumber / currentPowerOfTwo;

        // Füge digit an binärdarstellung an - nutzt int cast zu string bei + mit string
        bDarstellung = bDarstellung + digit;

        // Gehe zur nächstkleineren Zweierpotenz
        remainingNumber = remainingNumber - digit * currentPowerOfTwo;
        currentExp = currentExp - 1;
        currentPowerOfTwo = currentPowerOfTwo / 2;
    }

    return bDarstellung;
}
```

Step 2: If 2^currentExp >= number, then the next bit is 1, otherwise 0.

Exercise 2: Greatest Common Divisor

Schreiben Sie ein Programm “GGT.java”, das den grössten gemeinsamen Teiler (ggT) zweier ganzer Zahlen mithilfe des Euklidischen Algorithmus berechnet. Hierbei handelt es sich um eine iterative Berechnung, die auf folgender Beobachtung basiert:

1. Wenn x grösser als y ist, dann ist — sofern sich x durch y teilen lässt — der ggT von x und y gleich y;
2. andernfalls ist der ggT von x und y der gleiche wie der ggT von y und $x \,\%\, y$.

```
while (x <  y || x % y != 0) { //solange x kleiner als y oder x sich nicht durch y teilen lässt:
    //GGT noch nicht gefunden
    //update x und y für nächste Iteration
    int altY = y; // Zwischenspeichern von y

    y = x % y;
    x = altY;
}

//GGT gefunden (x >= y && x % y == 0)
System.out.println(y);
```

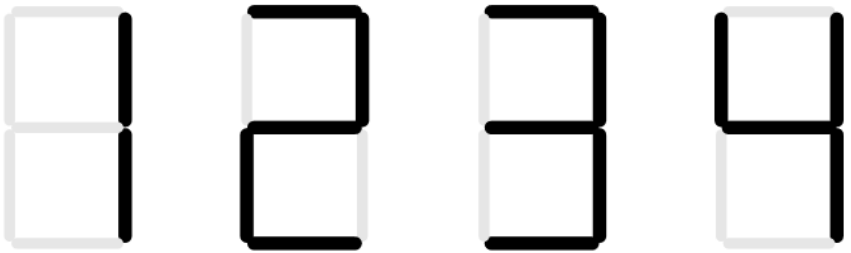
Negation of the condition in 1.

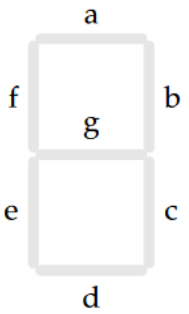
Exercise 3: Number Recognition

Für diese Aufgabe verwenden wir einen String um die erleuchtenden Segmente einer Sieben-segmentanzeige zu kodieren. Die Segmente sind, wie im Bild gezeigt, von a bis g nummeriert. Die Kodierung einer möglichen Anzeige ist ein String, in welchem der Buchstabe 'x' genau dann vorkommt, wenn das 'x'te Segment der Anzeige erleuchtet ist. Zum Beispiel wird die Zahl 2 kodiert durch 'abged'. Zur Einfachheit darf angenommen werden, dass kein Buchstabe mehr als einmal in der Kodierung vorkommt und dass nur die Zahlen 0 bis 9 kodiert werden.

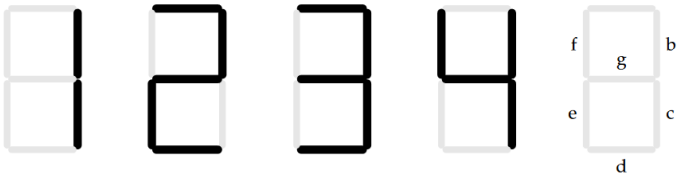
Schreiben Sie ein Programm "Zahlen.java", das einen String, der eine Anzeige kodiert, einliest und die kodierte Zahl als Integer ausgibt. Überlegen Sie wie viele IF Blöcke benötigt werden um jede Zahl zu erkennen.

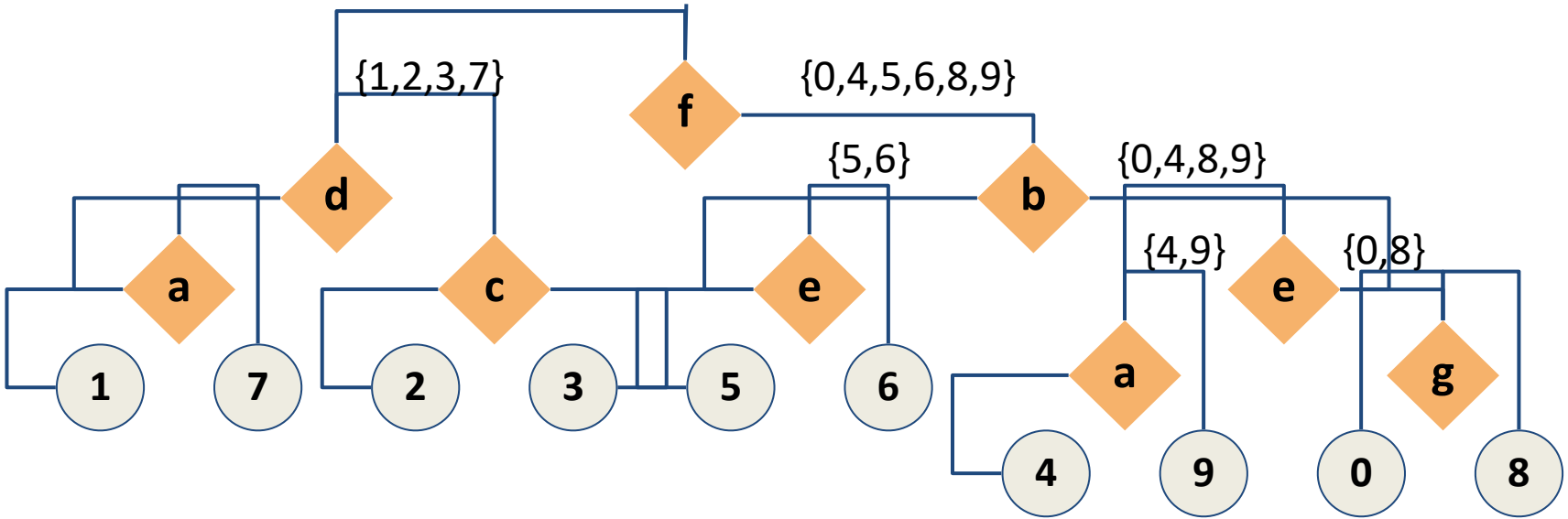
Tipp: Sie können `str.contains("a")` verwenden, um zu überprüfen, ob ein String `str` den Buchstaben 'a' enthält.





Exercise 3: Number Recognitio





Exercise 4: Calculations

2. Implementieren sie die Methode `Calculations.magic7(int a, int b)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn einer der Parameter 7 ist oder wenn die Summe oder Differenz der Parameter 7 ist. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele

   - `magic7(2,5)` gibt `true` zurück.
   - `magic7(7,9)` gibt `true` zurück.
   - `magic7(5,6)` gibt `false` zurück.

   Hinweis: Mit der Funktion `Math.abs(num)` können Sie den absoluten Wert einer Zahl `num` erhalten.

Exercise 4: Calculations

3. Implementieren Sie die Methode `Calculations.fast12(int z)`. Das Argument `z` ist nicht negativ. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn `z` nahe an einem Vielfachen von `12` ist. Eine Zahl `x` ist nahe an einer Zahl `y`, wenn eine der Zahlen um maximal `2` grösser oder kleiner ist als die andere Zahl. Ansonsten soll die Method `false` zurückgeben.

   - `fast12(12)` gibt `true` zurück.
   - `fast12(14)` gibt `true` zurück.
   - `fast12(10)` gibt `true` zurück.
   - `fast12(15)` gibt `false` zurück.

2. Implementieren Sie die Methode `Calculations.magic7(int a, int b)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn einer der Parameter 7 ist oder wenn die Summe oder Differenz der Parameter 7 ist. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele

   - `magic7(2,5)` gibt `true` zurück.
   - `magic7(7,9)` gibt `true` zurück.
   - `magic7(5,6)` gibt `false` zurück.

```java
public static boolean magic7(int a, int b) {  6 usages  zelleraa
    int sum = a + b;
    int diff1 = a - b;
    int diff2 = b - a;

    return a == 7 || b == 7 || sum == 7 || diff1 == 7 || diff2 == 7;
}
```

   Hinweis: Mit der Funktion `Math.abs(num)` können Sie den absoluten Wert einer Zahl `num` erhalten.

3. Implementieren Sie die Methode `Calculations.fast12(int z)`. Das Argument `z` ist nicht negativ. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn `z` nahe einem Vielfachen von 12 ist. Eine Zahl `x` ist nahe an einer Zahl `y`, wenn eine der Zahlen um maximal 2 grösser oder kleiner ist als die andere Zahl. Ansonsten soll die Methode `false` zurückgeben.

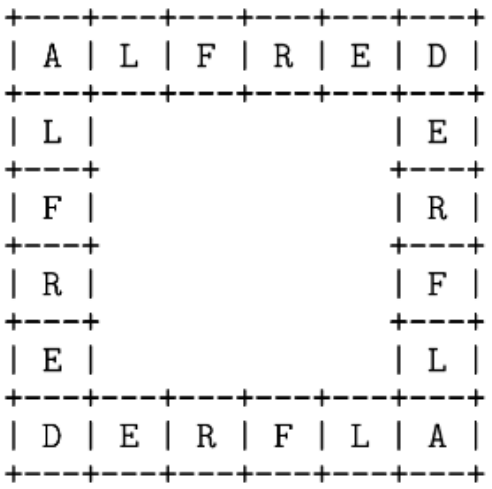
   Beispiele

   - `fast12(12)` gibt `true` zurück.
   - `fast12(14)` gibt `true` zurück.
   - `fast12(10)` gibt `true` zurück.
   - `fast12(15)` gibt `false` zurück.

```java
public static boolean fast12(int z) {  1 usage
    return Math.abs(z % 12) <= 2 || Math.abs(z % 12) >= 10;
}
```

Exercise 5: Scrabble

In dieser Aufgabe sollen Sie Scrabble-Steine legen, mittels ASCII-Art auf der Konsole. Vervollständigen Sie die Methode `drawNameSquare` in der Klasse `Scrabble`. Diese Methode nimmt einen Namen als String-Parameter und soll den Namen als in einem Quadrat angeordnete Scrabble-Steine auf der Konsole (`System.out`) ausgeben. Wenn z.B. der String `Alfred` übergeben wird, sollte folgendes Bild ausgegeben werden:



Preview

Exercise 1: Sieve of Eratosthenes

Schreiben Sie ein Programm "Sieb.java", das eine Zahl *limit* einliest und die Anzahl der Primzahlen, die grösser als 1 und kleiner oder gleich dem *limit* sind, ausgibt. Dazu ermitteln Sie in einem ersten Schritt alle Primzahlen, die kleiner oder gleich *limit* sind. Dieses Teilproblem können Sie mit dem Sieb des Eratosthenes lösen. Das Sieb des Eratosthenes findet Primzahlen bis n. Man betrachtet alle Zahlen von 2 bis n und streicht zuerst alle Vielfachen der ersten Zahl (2). Dann geht man zur nächsten ungestrichenen Zahl (3) und wiederholt das Streichen ihrer Vielfachen. Das macht man, bis man dies für alle Zahlen gemacht hat. Sie können ein `Boolean`-Array verwenden, um zu speichern, welche Zahlen Primzahlen sind und welche nicht. Übrig bleiben die Primzahlen. Danach können Sie die Anzahl der gefundenen Primzahlen anhand dieses Arrays bestimmen.

Beispiel: Für $limit = 13$ sollte Ihr Programm 6 ausgeben (Primzahlen: 2, 3, 5, 7, 11, 13).

Hinweis: Es ist nicht zwingend nötig von 2 bis n zu gehen. Von 2 bis $\sqrt{n}$ zu gehen reicht bereits aus, da eine Zahl $\leq n$ nicht einen Teiler grösser als $\sqrt{n}$ ausser sich selbst haben kann.

Exercise 2: Arrays

1. Implementieren Sie die Methode `ArrayUtil.zeroInsert(int[] x)` in der Datei "ArrayUtil.java". Die Methode nimmt einen Array `x` als Argument und gibt einen Array zurück. Der zurückgegebene Array soll die gleichen Werte wie `x` haben, ausser: Wenn eine positive Zahl direkt auf eine negative Zahl folgt oder wenn eine negative Zahl direkt auf eine positive Zahl folgt, dann wird dazwischen eine 0 eingefügt.

   Beispiele:

   - Wenn `x` gleich `[3, 4, 5]` ist, dann wird `[3, 4, 5]` zurückgegeben.
   - Wenn `x` gleich `[3, 0, -5]` ist, dann wird `[3, 0, -5]` zurückgegeben.
   - Wenn `x` gleich `[-3, 4, 6, 9, -8]` ist, dann wird `[-3, 0, 4, 6, 9, 0, -8]` zurückgegeben.

   Versuchen Sie, die Methode rekursiv zu implementieren.

Exercise 2: Arrays

2. Implementieren Sie die Methode `ArrayUtil.tenFollows(int[] x, int index)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn im Array `x` ab Index `index` der zehnfache Wert einer Zahl `n` direkt der Zahl `n` folgt. Dies muss nur für das erste Auftreten der Zahl `n` ab Index `index` im Array `x` geprüft werden. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele:

   - `tenFollows([1, 2, 20], 0)` gibt `true` zurück.
   - `tenFollows([1, 2, 7, 20], 0)` gibt `false` zurück.
   - `tenFollows([3, 30], 0)` gibt `true` zurück.
   - `tenFollows([3], 0)` gibt `false` zurück.
   - `tenFollows([1, 2, 20, 5], 1)` gibt `true` zurück.
   - `tenFollows([1, 2, 20, 5], 2)` gibt `false` zurück.

   Die `main` Methode in `ArrayUtil` gibt die oben genannten Beispielaufrufe sowie das entsprechende Ergebnis der jeweiligen Methode aus. Hiermit können Sie überprüfen, ob Ihre Implementierungen die richtigen Ergebnisse zurückliefern. In "ArrayUtilTest.java" im Ordner "test" in der Übungsvorlage finden Sie zusätzlich einige Unit-Tests für beide Methoden (für eine detaillierte Beschreibung zu automatisiertem Testen und der Ausführung solcher Tests siehe Aufgabe 3). Sie können die `main` Methode und die Tests beliebig abändern und/oder mit Ihren eigenen Inputs erweitern.

Exercise 3: 2D Arrays

Gegeben einer Matrix M, prüfen Sie zuerst ob diese eine $n \times n$ Matrix ist, deren Elemente positive ganze Zahlen sind. Danach prüfen Sie ob zusätzlich alle Zahlen kleiner gleich n^2 sind. Somit gilt nun $0 < m_{i,j} \leq n^2$. Prüfen Sie ebenfalls, ob die Elemente der Matrix jeweils genau einmal vorkommen, sprich ob $m_{x,y} = m_{p,q} \Rightarrow (x = p) \wedge (y = q)$ gilt. Wir sagen, dass die Matrix M *perfekt* ist, wenn zusätzlich alle Zeilensummen und Spaltensummen gleich sind (also $\sum_{k=0}^{k=n-1} m_{i,k} = \sum_{k=0}^{k=n-1} m_{j,k}$ für alle i, j und $\sum_{k=0}^{k=n-1} m_{k,i} = \sum_{k=0}^{k=n-1} m_{k,j}$ für alle i, j mit $0 \leq i, j < n$).

Vervollständigen Sie die Methode `boolean checkMatrix(int[][] m)` von der Klasse Matrix, so dass diese Methode `true` zurückgibt wenn die Input Matrix *perfekt* ist, und `false` sonst. Sie können davon ausgehen, dass der Parameter `m` nicht `null` ist. Alle anderen Eigenschaften müssen Sie selber testen. Eine Matrix ist nur perfekt, wenn alle genannten Eigentschaften gelten.

Testen Sie Ihr Programm ausgiebig - am besten mit JUnit - und pushen Sie die Lösung vor dem Abgabetermin. Wir haben Ihnen einen JUnit Test in der Klases `MatrixTest` bereits erstellt.

Exercise 4: Testing with JUnit

Purpose of the program:

- Output the weekday of a date (after 01.01.1900)
  Example:13.10.2017 → Friday
  However, incorrectly outputs *“The 13.10.2017 is a Sunday”* aus.
- Takes leap years into account (“Leap year”)

How it works:

1. Checks whether the date is OK
2. Counts the days from 1.1.1900 to the entered date
3. Weekday = days % 7

Exercise 4: Testing with JUnit

Tests in `PerpetualCalendarTest.java`

- Individual tests check return values of individual methods of the program `PerpetualCalendar.java`
- Tests should use interesting parameters for the methods
- Example `testCountDaysInYear()`:
- `assertEquals(366, PerpetualCalendar.countDaysInYear(1904));`
- 1904 is a leap year, so `countDaysInYear()` should return 366 days

DEMO

Exercise 5: Matching Numbers

Implementieren Sie die Methode `Match.matchNumber(long A, int M)`. Die Methode soll für eine Zahl `A` und eine nicht-negative drei-stellige Zahl `M` die Position von `M` in `A` zurückgeben. Sei `M` eine Zahl mit den Ziffern $M_2M_1M_0$ (das heisst, es gilt $M = M_0 + 10 \cdot M_1 + 100 \cdot M_2$), wobei jede Ziffer 0 sein kann. Zusätzlich sei `A` eine Zahl, sodass A_i die i-te Ziffer von `A` ist (das heisst, es gilt $|A| = \sum_i 10^i \cdot A_i$), wobei `A` unendlich viele führende Nullen hat. Die Position von `M` in `A` ist die kleinste Zahl j, sodass $A_j = M_0$ und $A_{j+1} = M_1$ und $A_{j+2} = M_2$ gilt. Die Methode soll `-1` zurückgeben, falls es kein solches j gibt.

Beispiele:

`matchNumber(32857890, 789)` soll `1` zurückgeben.
`matchNumber(37897890, 789)` soll `1` zurückgeben.
`matchNumber(1800765, 7)` soll `2` zurückgeben.
`matchNumber(1800765, 8)` soll `-1` zurückgeben (die drei Ziffern von `8` sind `008`).
`matchNumber(75, 7)` soll `1` zurückgeben (da `007` and Position `1` von `0075` ist).

Exercise 5: Matching Numbers

Implementieren Sie die Berechnung in der Methode `int matchNumber(long A, int M)`, welche sich in der Klasse `Match` befindet. Die Deklaration der Methode ist bereits vorgegeben. Sie können davon ausgehen, dass $0 \leq \texttt{M} < 1000$ gilt.

In der `main` Methode der Klasse `Match` finden Sie die oberen Beispiele als kleine Tests, welche Beispiel-Aufrufe zur `matchNumber`-Methode machen und welche Sie als Grundlage für weitere Tests verwenden können. In der Datei `MatchTest.java` geben wir die gleichen Tests zusätzlich auch als JUnit Test zur Verfügung. Sie können diese ebenfalls nach belieben ändern. Es wird *nicht* erwartet, dass Sie für diese Aufgabe den JUnit Test verwenden.

Tipp: Die Methode `Integer.toString(int i)` wandelt einen Integer in einen String um.